

LOW-LEVEL DESIGN

Jeeran Network

Schema, functions, API contracts, and sequence-level detail for the implemented system.

Version 1.0 · August 2026

1. Database Schema — Core Tables

Table	Key columns	Purpose
<code>businesses</code>	<code>id</code> , <code>owner_id</code> , <code>category</code> , <code>latitude/longitude</code> , <code>ad_credits</code> , <code>is_subscription_active</code> , <code>is_frozen</code> , <code>last_milestone_tier</code> , <code>unseen_milestone_tier</code> , <code>unseen_milestone_bonus</code>	One row per shop; the credit ledger balance lives here
<code>campaigns</code>	<code>id</code> , <code>creator_type</code> , <code>creator_id</code> , <code>title</code> , <code>bid_per_view</code> , <code>is_active</code> , <code>start_date</code> , <code>end_date</code> , <code>updated_at</code>	One active campaign per creator (enforced constraint); <code>updated_at</code> is trigger-maintained and drives content-change detection
<code>wallet_members</code>	<code>id</code> , <code>device_id</code> , <code>origin_business_id</code> , <code>home_lat/lng</code> , <code>google_object_id</code> , <code>last_notified_offers</code> (jsonb), <code>signup_bonus_paid</code>	One row per customer pass. <code>origin_business_id</code> is permanent (first-touch)
<code>redemptions</code>	<code>member_id</code> , <code>business_id</code> , <code>offer_id</code> , unique on (member, offer)	The record that both blocks re-claiming and drives promotion score
<code>wallet_member_touchpoints</code>	<code>member_id</code> , <code>business_id</code> , <code>scanned_at</code> , <code>scan_ip</code>	Append-only log of every scan, first-touch or not
<code>rescan_credit_events</code>	<code>touchpoint_id</code> , <code>business_id</code> , <code>role</code> ('scanned' 'origin'), <code>amount</code>	One row per actual rescan-credit payout — makes the daily cap a plain count query
<code>promotion_settings</code>	<code>silver/gold/platinum_threshold</code> , <code>bid_tiebreak_range</code> , <code>silver/gold/platinum_milestone_bonus</code>	Single-row, admin-editable config every tier-aware function reads
<code>offer_notifications</code>	<code>member_id</code> , <code>business_id</code> , <code>offer_id</code> , unique on (member, offer)	Reach tracking — feeds the notified→redeemed conversion metric

Legacy tables (`scans_and_claims` , the one-off claim flow) remain for the original per-offer coupon model, which still operates in parallel for Apple Wallet and one-time links. They deliberately do not feed promotion score.

2. Key Postgres Functions

Function	Signature	Behavior
<code>get_top_ads</code>	(host_business_id, limit)	Scan-page matchmaking. Excludes host's own campaigns, same-category competitors, linked technicians. Orders by bid desc, then a 5-minute rotating hash tiebreak.
<code>nearby_active_offers</code>	(lat, lng, radius_km, limit)	Wallet geo-push matchmaking. Orders by a bid-bucket (close bids grouped by <code>bid_tiebreak_range</code>) desc, then promotion tier rank desc, then exact bid desc, then distance asc. Also returns each offer's coordinates and <code>updated_at</code> for content-change detection.
<code>browse_businesses_by_category</code>	(category, limit)	Public directory listing. Orders purely by redemption score — no bid dimension exists in this context, so tier is the primary (and only) signal by design.
<code>promotion_level_for_score</code>	(score) → text	Reads thresholds from <code>promotion_settings</code> live — <code>stable</code> , not <code>immutable</code> , so an admin edit takes effect immediately everywhere.
<code>claim_ad_credit_transaction_membership</code>	(member_id, offer_id, redeeming_business_id)	<code>FOR UPDATE</code> -locks the campaign row; transfers the offer's bid from the redeeming shop to the member's <code>origin_business_id</code> ; a no-op (not an error) if they're the same shop.
<code>check_and_award_milestone_bonus</code>	(business_id) → (crossed, new_tier, bonus_awarded)	<code>FOR UPDATE</code> -locks the business row; recomputes score/tier; pays the bonus for the newly-reached tier only if strictly higher than <code>last_milestone_tier</code> ; sets the unseen-announcement pair atomically in the same statement.
<code>pay_signup_bonus</code> / <code>pay_rescan_credit</code>	(business_id, amount)	Flat, unconditional credit — not a transfer, so no locking needed beyond the caller's own cap check.

check_and_award_milestone_bonus — core logic

```

select last_milestone_tier into v_old_tier
  from businesses where id = p_business_id for update;

select count(*) into v_score from redemptions where business_id = p_business_id;
v_new_tier := promotion_level_for_score(v_score);

if promotion_level_rank(v_new_tier)
  <= promotion_level_rank(coalesce(v_old_tier, 'bronze')) then

```

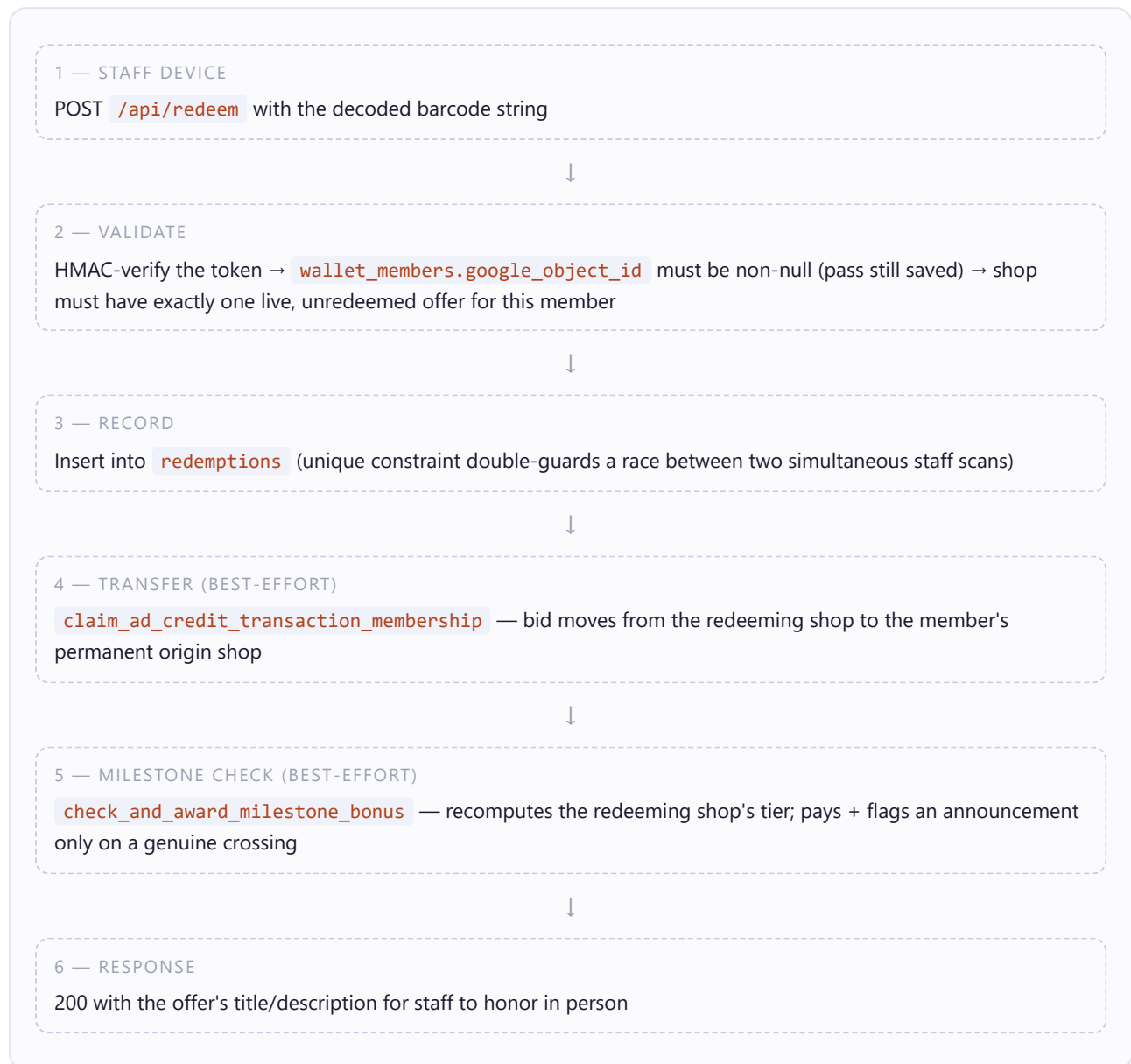
```
    return query select false, v_new_tier, null::numeric;  -- nothing to do
    return;
end if;

-- look up this tier's bonus, pay it, record the crossing
update businesses set ad_credits = ad_credits + v_bonus,
    last_milestone_tier = v_new_tier,
    unseen_milestone_tier = v_new_tier,
    unseen_milestone_bonus = v_bonus
where id = p_business_id;
```

3. API Surface

Route	Method	Auth	Purpose
/api/wallet/membership/create	POST	Anonymous (device cookie)	Create-or-reuse the customer's pass; pays signup bonus or fires rescan-credit depending on new/returning
/api/redeem	POST	Shop session	Validates + records a redemption; triggers ad-credit transfer and milestone check
/api/campaigns , /api/campaigns/[id]	POST / PATCH	Shop session	Create / edit / toggle a campaign; edit and toggle both fire the geo-push trigger
/api/cron/wallet-refresh	GET	Bearer CRON_SECRET	Daily safety-net sweep across every registered member
/api/dashboard/leaderboard	GET	Shop session	Top shops by genuine new-customer count, week or month
/api/dashboard/dismiss-milestone	POST	Shop session	Clears the unseen-milestone announcement pair for the caller's own shop
/api/admin/settings	GET / PATCH	None (tracked gap)	Read/write tier thresholds, bid-tiebreak range, milestone bonus amounts
/api/admin/shops/[id]/freeze	PATCH	None (tracked gap)	Freeze/unfreeze a shop

4. Sequence — Redemption With All Downstream Effects



Steps 4 and 5 are independent of each other and of the customer-facing response — a failure in either is logged for reconciliation, never surfaced as a redemption failure.

5. Sequence — Existing Member Rescans a QR Stand

1

POST `/api/wallet/membership/create`, device cookie matches an existing `wallet_members` row



2 — COOLDOWN CHECK

Query `wallet_member_touchpoints` for this (member, scanned business) pair within the last 24h — *before* inserting the new row, or it would always find itself



3 — LOG, ALWAYS

Insert the new touchpoint (with request IP) regardless of the cooldown result



4 — AWARD, ONLY IF OUTSIDE COOLDOWN

Two independent, cap-checked calls: `RESCAN_CREDIT` to the scanned business, `ORIGIN_RESCAN_CREDIT` to the permanent origin business — naturally both target the same business when they're equal, no special-case branch needed

Each award call independently re-checks that business's own daily cap via a count against `rescan_credit_events` — a shop already at its cap for the day is simply skipped for that role, while the other award can still succeed.

6. Notable Implementation Decisions

Decision	Reasoning
Barcode is <code>memberId + '.' + HMAC(memberId)</code> , not a URL	The legacy one-off flow's barcode is a URL by design (any camera app can open it); the persistent pass barcode must not be, since it needs a dedicated staff-side scanner to have any effect — verified live that a generic camera app just shows the raw string
Google Wallet field is <code>merchantLocations</code> , not <code>locations</code>	Confirmed against a live API rejection before shipping — <code>locations</code> is the correct name for Apple's format only
Lock-screen notification requires <code>addMessage + messageType: TEXT_AND_NOTIFY</code>	Embedding an untyped message in the general object PATCH updates the card silently but never actually notifies — confirmed via a real-device test showing no notification, then corrected against Google's Message reference
Tier is a tiebreaker within a bid bucket, not a primary sort key	An earlier iteration made tier fully primary (a large ranking-behavior change); revised to preserve the bid auction's incentive while still giving loyal shops an edge in genuinely contested (close-bid) slots
Content-change detection compares <code>{offer_id: updated_at}</code> , not just <code>offer_id</code> sets	A same-offer edit (title/description/small bid change) doesn't change which IDs are in a member's top-5, so an ID-only diff silently missed it — confirmed live before the fix, then after